Module Definitions

module *name*
  body
end

A module is a class that cannot be instantiated. Like a class, its body is executed during definition, and the resulting Module object is stored in a constant. A module may contain both class and instance methods and may define constants and class variables. (A module can also reference instance variables, but those are dependent on the module being added to a class, see the next section.)

As with classes, a module's class methods (sometimes called *module methods*) are invoked using the Module object as a receiver, and constants are accessed using the :: scope resolution operator. The name in a module definition may optionally be preceded by the names of enclosing class(es) and/or module(s).

```
CONST = "outer"
module Mod
  CONST = 1

  def self.method1    # module method
    CONST + 1
  end
end

module Mod::Inner
  def self.method2
    CONST + " scope"
  end
end

Mod::CONST          # => 1
Mod.method1         # => 2
Mod::Inner::method2 # => "outer scope"
```

Mixins: Including Modules

class|module *name*
  include *expr*
end

A module may be included within the definition of another module or class using the include method. The module or class definition containing the include gains access to the constants, class variables, and instance methods of the module it includes.

If a module is included within a class definition, the module's constants, class variables, and instance methods are made available as if the module was an additional anonymous (and inaccessible) superclass for that class. Objects of the class will respond to messages sent to the module's instance methods. Calls to methods not defined in the class will be passed to the module(s) mixed into the class before being passed to any parent class. A module may define an initialize method, which will be called upon the creation of an object of a class that mixes in the module if either the class does not define its own initialize method or the class's initialize method invokes super.